

Python Dosyalama İşlemleri ve SQL Bağlantısı

Mehmet Hakan Satman (PhD.)

April 15, 2022

İçindekiler

1.1	open, write ve close	2
1.2	open, read ve close	3
1.3	CSV dosyalarının okunması	5
1.4	Nesnelerin Serileştirilmesi	7
1.5	Sqlite3 Veritabanı Bağlantısı	8
1.5.1	Çalışma soruları	12

1.1 open, write ve close

Basit dosyalama işlemleri için `file` fonksiyonundan faydalanabiliriz. `file` ile bir dosyaya yazılabilir, satırlar tek tek okunabilir veya tüm dosya içeriği tek seferde belleğe yüklenebilir. Aşağıdaki Python kodu ile bir dosya açılmakta içine *Merhaba Dünya!* ifadesi yazdırılmaktadır:

```
dosya = open("dosyam.txt", "w")
dosya.write("Merhaba Dünya!")
dosya.close()
```

Kod içinde `open` fonksiyonu ile belirtilen isimde bir dosya açılır. İkinci argüman `w` olarak verilmiştir. Böylece dosyanın yazılmak için açıldığı anlaşılır. İşletim sistemi düzeyinde bir dosyanın okunmak, yazılmak veya hem okunup hem de yazılmak üzere açıldığı meselesi önem kazanır. Bu argümanın alabileceğini birçok değerden bazıları şunlardır:

- **w**: Dosya yalnızca yazılmak için açılır. Yazma işlemi dosyanın başından itibaren gerçekleştirilir.
- **r**: Dosya yalnızca okunmak için açılır.
- **a**: Dosya yazılmak için açılır ancak bu kez yazılma işlemi var olan dosyanın sonundan itibaren gerçekleştirilir.

`dosya` nesnesine ait olan `write` fonksiyonu ile içeriğe belirtilen ifade eklenmiştir. Dosya açma modu `w` olarak verildiği için yazma işlemi dosyanın başından itibaren gerçekleştirilir.

Son olarak dosya nesnesine ait olan `close` fonksiyonu ile dosya kapatılır. Oluşan `dosyam.txt` adlı dosyanın içeriği bir metin editörü ile izlenebilir:

Merhaba Dünya!

Aynı kod tekrar çalıştırıldığında `dosyam.txt` adlı dosyanın içeriği

Merhaba Dünya!

olur çünkü `w` modu her dosya açma işleminde dosya imlecini dosyanın başına getirmektedir. Örnek kodda dosya açma modu `a` (Append) ile değiştirilirse aşağıdaki şekilde olur:

```
dosya = open("dosyam.txt", "a")
dosya.write("Merhaba Dünya!")
dosya.close()
```

Bu kod birkaç kez üst üste çalıştırılırsa `dosyam.txt` adlı dosyanın içeriği aşağıdaki şekilde olur:

Merhaba Dünya!Merhaba Dünya!Merhaba Dünya!Merhaba Dünya!Merhaba Dünya!

Kayıtlar arasında `enter` tuşunun da yer alması isteniyorsa yazılacak her bir satırın sonuna `n` ifadesi eklenebilir:

```
dosya = open("dosyam.txt", "w")

dosya.write("Merhaba Dunya!\n")
dosya.write("Hoşca kal Dunya!\n")

dosya.close()
```

Yukarıdaki örnekte dosya `w` ile açıldığından belirtilen içerik silinir. Ayrıca dosyaya iki satırlık veri enter ile ayrılmış şekilde yazılır. Kod çalıştırıldığında dosyam.txt adlı dosyanın içeriği aşağıdaki gibi olur:

```
Merhaba Dunya!
Hoşca kal Dunya!
```

1.2 open, read ve close

`open` fonksiyonu belirtilen dosya adı ve dosya açma moduyla çağırıldığında bir nesne döndürür:

```
dosya = open("dosyam.txt", "w")

tip = type(dosya)
print(tip)

dosya.close()
```

Bu program çalıştırıldığında alınan

```
<class '_io.TextIOWrapper'>
```

çıktısıyla birlikte dosya'nın tipinin `io.TextIOWrapper` olduğu görülür. Öncesinde kullandığının `write` ve `close` fonksiyonları bu sınıf içinde tanımlanmış üye fonksiyonlarıydı. Bu sınıfın tanımladığı diğer fonksiyonlar ile diğer bazı temel dosyalama işlemleri gerçekleştirilebilir. Örneğin içeriği

```
Merhaba Dunya!
Hoşca kal Dunya!
```

olan dosyayı okumaya çalışalım:

```
dosya = open("dosyam.txt", "r")

icerik = dosya.read()

print(icerik)

dosya.close()
```

Bu kodun çıktısı aşağıdaki gibi olur:

Merhaba Dünya!
Hoşça kal Dünya!

`read` fonksiyonu ile dosyanın tüm içeriği `icerik` adlı değişkende saklanır. Bu işlem çok büyük dosyalarda kullanılamaz çünkü okunmak istenen dosya boyutu mevcut belleğin üstünde olabilir. Bazen dosyayı parça parça okuyup bu parçalar üstünde bazı işlemler gerçekleştirmek isteriz. Dosyadan tek bir satır okumak için

```
dosya = open("dosyam.txt", "r")

icerik = dosya.readline()

print(icerik)

dosya.close()
```

kullanılabilir. Buradaki `readline` fonksiyonu dosya adlı nesneye aittir ve yukarıda belirttiğimiz sınıf içinde tanımlanmıştır. Bu kod çalıştırıldığında ekranda

Merhaba Dünya!

görülür çünkü yalnızca ilk satır okunup dosya kapatılmıştır. Dosya izin verdikçe satırlar bir döngü ile de okunabilir:

```
dosya = open("dosyam.txt", "r")

while True:
    icerik = dosya.readline()
    if len(icerik) == 0:
        break
    print(icerik)

dosya.close()
```

Yukarıdaki döngü ile dosyadan her seferinde yalnızca bir satır veri okunmakta, eğer okunan verinin uzunluğu sıfırsa döngüden çıkmaktadır. Aşağıdaki çıktı elde edilir:

Merhaba Dünya!

Hoşça kal Dünya!

Bu satırların hepsi tek seferde bir liste içinde yer alacak şekilde okunabilirdi:

```
dosya = open("dosyam.txt", "r")

icerikler = dosya.readlines()

for satir in icerikler:
    print(satir)

dosya.close()
```

Dosyanın içeriğini bu şekilde okumanın, `read` ile tek seferde okumaktan farkı şudur: `readlines` tüm satırları bir liste içinde saklar. `text` ise her şeyi tek bir string içinde saklar ve tutulan veri satır satır ayrıştırılmış durumda değildir.

1.3 CSV dosyalarının okunması

Bir CSV (Comma seperated values) dosyasının aşağıdaki şekilde kaydedilmiş olduğunu varsayalım:

```
X; Y
1; 10
2; 15
3; 20
4; 21
5; 25
6; 19
7; 30
8; 32
9; 30
10; 35
```

`dosyam.csv` olarak kaydedilen bu CSV dosyası için aşağıdaki Python kodunun çalıştırıldığını düşünelim:

```
dosya = open("dosyam.csv", "r")

basliklar = dosya.readline()

basliklistesi = basliklar.split(";")

print(basliklistesi)

dosya.close()
```

`dosyam.txt` dosyası okuma modunda açılmış ve dosyadan tek bir satır okunmuştur. Biz verinin ilk satırında değişken adlarının yer aldığını biliyoruz. Veriden ilk satırı okuyup bu satırın `basliklar` degiskeninde tutulmasını sağladık. Başlık satırı noktalı virgül ile ayrılmış başlık adlarını içermektedir. Bu yüzden `basliklar` adındaki ve string tipindeki nesne üzerinden `split` fonksiyonunu çağırdık. Split fonksiyonu bir string ifadeyi belirtilen ifade için parçalara ayırır ve bu parçaların bir listesini döndürür. Sonrasında ise `basliklistesi` adlı liste print ile ekrana yazılır. Kod çalıştırıldığında aşağıdaki çıktı elde edilir:

```
['X', ' Y\n']
```

Ayrıca CSV dosyasında her bir satır birer enter içerdiğinde `readline` fonksiyonunun bu enter karakterlerini de aldığını görüyoruz. Öncesinde enter karakterlerini silip sonrasında `split` fonksiyonunu kullanabilirdik:

```
dosya = open("dosyam.txt", "r")

basliklar = dosya.readline()

basliklistesi = basliklar.replace("\n", "").split(";")

print(basliklistesi)

dosya.close()
```

Bu programda

```
basliklistesi = basliklar.replace("\n", "").split(";")
```

olan satırda basliklar satırından enter karakteri silinir ve sonrasında oluşan satır noktalı virgül ile parçalanır. Çıktı aşağıdaki gibi olur:

```
['X', ' Y']
```

Şimdi programı tamamlayıp verinin geri kalanını da okuyabiliriz:

```
dosya = open("dosyam.txt", "r")

basliklar = dosya.readline()

basliklistesi = basliklar.replace("\n", "").split(";")

csv = []

satirlar = dosya.readlines()
for satir in satirlar:
    csv.append(satir.replace("\n", "").split(";"))

print(basliklistesi)
print(csv)
dosya.close()
```

Program çalıştırıldığında aşağıdaki çıktılar elde edilir:

```
['X', ' Y']
[
    ['1', ' 10'],
    ['2', ' 15'],
    ['3', ' 20'],
    ['4', ' 21'],
    ['5', ' 25'],
    ['6', ' 19'],
    ['7', ' 30'],
    ['8', ' 32'],
    ['9', ' 30'],
    ['10', ' 35']
]
```

Veriler her satırda iki string ifade yer alacak şekilde otomatik olarak oluşturulur. Buradaki veri yapısının iki boyutlu bir liste olduğu görülebilir. Satır elemanlarına çift indis ile ulaşılabilir:

```
print(csv[2][0])
print(csv[2][1])
print(csv[2][0:])
```

csv adlı liste 2 boyutludur. csv[2][0] ile üçüncü satır ve birinci sütun elemanına erişilir. csv[2][1] ile üçüncü satır ve ikinci sütun elemanına erişilir. csv[2][0:] ile üçüncü satır elemanı bir liste olarak elde edilir. Kod çalıştırıldığında aşağıdaki değerler elde edilir:

```
3
20
['3', '20']
```

1.4 Nesnelerin Serileştirilmesi

Bir Python nesnesinin bir dosyada saklanması veya bir dosyadan okunmasını isteyebiliriz. class ile oluşturulmuş bir nesnenin bir dosyada belirli formatlarla kaydedilmesine nesne serileştirme adı verilir. Aşağıdaki kodu inceleyelim:

```
class Person:
    def __init__(self, name, surname, age):
        self.name = name
        self.surname = surname
        self.age = age
```

Person adlı sınıftan aşağıdaki gibi bir nesne oluşturulur:

```
p1 = Person("Zeki", "Yel", 42)
```

p1 adlı nesne Person tipinde bir veri nesnesini gösterir. Bu veri nesnesi Person'a ait init fonksiyonu sayesinde dışarıdan alınan name, surname ve age değerlerine sahip olur. Şimdi Person sınıfına aşağıdaki fonksiyonu da ekleyelim:

```
class Person:
    def __init__(self, name, surname, age):
        self.name = name
        self.surname = surname
        self.age = age

    def write(self):
        myfile = open(self.name + "-" + self.surname + ".txt", "w")
        myfile.write(self.name + ";" + self.surname + ";" + str(self.age))
        myfile.close()
```

Sınıfa eklenen yeni fonksiyon, söz konusu nesnenin name ve surname değerlerinin bir bileşimi ile bir dosya adı ve dosya oluşturur. Bu dosyaya ilgili nesnenin içeriğini noktalı virgül ayrıçlarıyla yazar ve son olarak dosyayı kapatır. Söz konusu sınıfı şimdi aşağıdaki şekilde uygulayalım:

```
p1 = Person("Zeki", "Yel", 42)
p1.write()
```


p1 nesnesi üzerinden `write` fonksiyonu çağırılır çağırılmaz geçerli klasörde `zeki-yel.txt` adlı bir dosya oluşacaktır. Bu dosyanın içeriği aşağıdaki gibi olur:

```
Zeki;Yel;42
```

1.5 Sqlite3 Veritabanı Bağlantısı

Python ile SQLite veritabanlarına `sqlite3` paketi yardımıyla bağlanabiliriz. Bir veritabanına bağlantı ve bağlantının kapatılması aşağıdaki şekilde gerçekleştirilebilir:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

mydb.close()
```

Örnek kodda `mydatabase.db` adlı dosyaya bir bağlantı gerçekleştirilir. Adı belirtilen dosya yoksa ilk defa oluşturulur. Varsa, var olan dosyaya bir bağlantı gerçekleştirilir. Bu veritabanı ile ilgili işlemler, örnekte, `mydb` değişkeni aracılığı ile işleme tabi tutulur. Son olarak da `close` fonksiyonu ile bağlantı sonlandırılır. Şimdi söz konusu veritabanı içinde bir tablo oluşturalım:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

mydb.execute(
    """
    create table Ogrenciler
    (id text, name text, surname text)
    """
)

mydb.close()
```

Örnekte kullanılan `"""` karakteri birden fazla satıra sahip string değerleri tanımlanmak için kullanılmıştır. `mydb` üzerinden çağırılan `execute` fonksiyonu ile `Ogrenciler` adında yeni bir tablo oluşturulur. Bu tabloya yeni veriler ekleyelim:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

c = mydb.cursor()

c.execute("insert into Ogrenciler (id, name, surname) values ('12564545', '
                                         Cemil', 'Demir')")

mydb.commit()

mydb.close()
```

Yukarıdaki Python kodunda öncelikle veritabanına bir bağlantı sağlanır ve bu bağlantıya mydb değişkeni ile erişilir. cursor() fonksiyonu ile veritabanı için bir kursör nesnesine ulaşılır. Verileri yazma işi doğrudan veritabanı üzerinden değil, bu kursör üzerinden gerçekleştirilir. Kursör nesnesi üzerinden çağrılan execute fonksiyonu ile SQL'e ait INSERT INTO cümleceği (Statement) çalıştırılır. Sonrasında mydb üzerinden çağrılan commit fonksiyonu ile kursör üzerinde yapılan değişiklikler veritabanına yansıtılır. Son olarak da veritabanı bağlantısı sonlandırılır. Tablo içeriği aşağıdaki şekilde değilmiş olacaktır:

12564545|Cemil|Demir

Şimdi veritabanına birkaç kayıt daha ekleyelim:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

c = mydb.cursor()

c.execute("insert into Ogrenciler (id, name, surname) values ('12564546', 'Cemile', 'Ay')")
c.execute("insert into Ogrenciler (id, name, surname) values ('12564547', 'Hasan', 'Bulut')")
c.execute("insert into Ogrenciler (id, name, surname) values ('12564548', 'Cavit', 'Turuncu')")
c.execute("insert into Ogrenciler (id, name, surname) values ('12564549', 'Zuhal', 'Pul')")

mydb.commit()

mydb.close()
```

Aşağıdaki Python kodu ile tüm kayıtları okuyabiliriz:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

c = mydb.cursor()

c.execute("select * from Ogrenciler")

satirlar = c.fetchall()

for satir in satirlar:
    print(satir)

mydb.close()
```

Örnekte görüldüğü gibi sorgudan dönen tüm satırlara fetchall fonksiyonu ile ulaşılmıştır. Bu satırların herbirine sırasıyla erişmek için bir for döngüsü kullanılmıştır. Döngü sırasında satırlar değişkenindeki her bir satır ekrana yazdırılır. Programın çıktısı aşağıdaki gibi gerçekleşir:

```
('12564545', 'Cemil', 'Demir')
('12564546', 'Cemile', 'Ay')
('12564547', 'Hasan', 'Bulut')
('12564548', 'Cavit', 'Turuncu')
('12564549', 'Zuhal', 'Pul')
```

Sorgudaki ilk satıra erişmek için fetchone fonksiyonu kullanılabilir:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

c = mydb.cursor()

c.execute("select * from Ogrenciler")

satir = c.fetchone()
print(satir)

mydb.close()
```

Çıktı aşağıdaki gibi olur:

```
('12564545', 'Cemil', 'Demir')
```

Her bir satır tek tek kayıt sonuna kadar bir döngü içinde okunabilir:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

c = mydb.cursor()

c.execute("select * from Ogrenciler")

while True:
    satir = c.fetchone()
    if satir is None:
        break
    print(satir)

mydb.close()
```

Çıktı aşağıdaki gibi olur:

```
('12564545', 'Cemil', 'Demir')
('12564546', 'Cemile', 'Ay')
('12564547', 'Hasan', 'Bulut')
('12564548', 'Cavit', 'Turuncu')
('12564549', 'Zuhal', 'Pul')
```

Kayıtlar okunurken her bir alana indis numarası ile erişilebilir:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")

c = mydb.cursor()

c.execute("select * from Ogrenciler")

satirlar = c.fetchall()

for satir in satirlar:
    print("Okunan satir: " + satir[1] + " " + satir[2])

mydb.close()
```

```
Okunan satir: Cemil Demir
Okunan satir: Cemile Ay
Okunan satir: Hasan Bulut
Okunan satir: Cavit Turuncu
Okunan satir: Zuhal Pul
```

Veritabanından okunan verilerin Excel veya R'dan okunabilmesi için bir CSV dosyasında saklanmasını isteyebiliriz. Aşağıdaki kod ile Ogrenciler tablosundaki Excel'de açılabilme üzere bir metin dosyasında saklanır:

```
import sqlite3

mydb = sqlite3.connect("mydatabase.db")
myfile = open("mydatabase.csv", "w")

c = mydb.cursor()
c.execute("select * from Ogrenciler")

satirlar = c.fetchall()

myfile.write("Numara;Ad;Soyad\n")

count = 0

for satir in satirlar:
    myfile.write(satir[0] + ";" + satir[1] + ";" + satir[2] + "\n")
    count += 1

print(str(count) + " satir yazildi.")
myfile.close()
mydb.close()
```

Kod çalıştırıldığında aşağıdaki ekran çıktısı elde edilir:

```
5 satir yazildi.
```

Ekran çıktısının yanında mydatabase.csv adlı bir dosya oluşturulmuştur. Bu dosya Excel veya metin editöründe açıldığında aşağıdaki içeriğe sahip olduğu görülür:

```
Numara;Ad;Soyad  
12564545;Cemil;Demir  
12564546;Cemile;Ay  
12564547;Hasan;Bulut  
12564548;Cavit;Turuncu  
12564549;Zuhal;Pul
```

1.5.1 Çalışma soruları

Oğrenciler tablosından aşağıdaki sorgulamaları gerçekleştiriniz:

1. Adı C ile başlayan öğrencileri listeleyiniz.
2. Soyadı Ay olan öğrencileri listeleyiniz.
3. Adı Zuhal ve soyadı Pul olan öğrencinin numarasını gösteriniz.
4. Hasan Bulut adlı öğrencinin numarasını 0505440011 olarak değiştiriniz.
5. Cemile Ay'ın kaydını siliniz.
6. Tüm tabloyu temizleyiniz.